FC FarmConnect Ghana

SYSTEM ANALYSIS & DESIGN DOCUMENTATION

Design Documentation

Architecture, UML Diagrams, Database Design & UI Wireframes for the Delivered System

Every diagram in this document depicts the system as it was actually built and deployed — module names, routes, and data-model fields are traced directly from source (Prisma schema, Express routers, React routes), not redrawn from the original proposal. Where a diagram differs from what the SRS v1.0 originally imagined, the difference is explained inline.

Course	**CSCD602 — Advanced Software Engineering, Capstone Project**
Group	**Angry Nerds**
Project title	**FarmConnect Ghana**
Document version	**1.0**
Date	**12 August 2026**
Individual submission by	**George Boadu Junior — 22427354**

Companion document to the Software Requirements Specification. See the SRS "Group Information" section for the full roster.

Table of Contents

1. Design Approach & Notation

Diagrams below use standard UML notation (use-case, class, sequence, state) plus one informal deployment/architecture diagram and a set of low-fidelity UI wireframes. Every diagram was produced *from* the delivered source tree — routes, models, and status enums are copied verbatim from `apps/api/prisma/schema.prisma`, `packages/shared/src`, and the Express routers — rather than being an idealised plan drawn before implementation. Colour coding used consistently across diagrams:

	FarmConnect-owned component (frontend, backend module, actor role)
	Persistent data store (PostgreSQL, Redis)
	External third-party service (Gemini, OpenWeatherMap, SMS gateways)
	Terminal/negative state or blocked path (cancelled, rejected, suspended)

2. System & Deployment Architecture

Three tiers — client, API, data/external — matching SRS §7, shown here with the actual protocols and providers used at runtime.

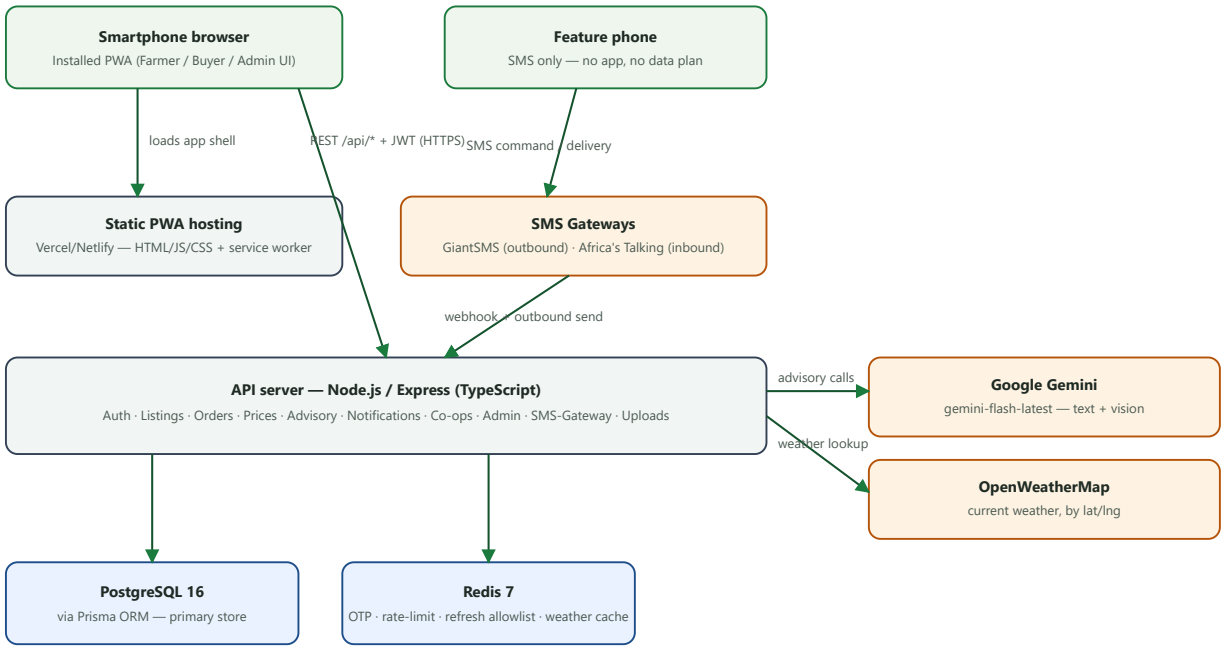

Figure 2.1 — Deployment/architecture view. Clients never talk to Postgres/Redis/Gemini/OpenWeatherMap directly; every external and data call is mediated by the single Express API, which is the only component holding credentials for any downstream service.

WHY ONE API, NOT MICRO-SERVICES

NFR-M1 asks for modules "organised to simplify... future extraction into independent services" — not that they *be* separate services today. A single deployable API with clean module boundaries (see §3) gets the maintainability benefit without the operational cost of running/monitoring nine separate services for a capstone-scale user base.

3. Component View

A logical (code-module) view, distinct from the physical deployment view above: what depends on what at the package/module level.

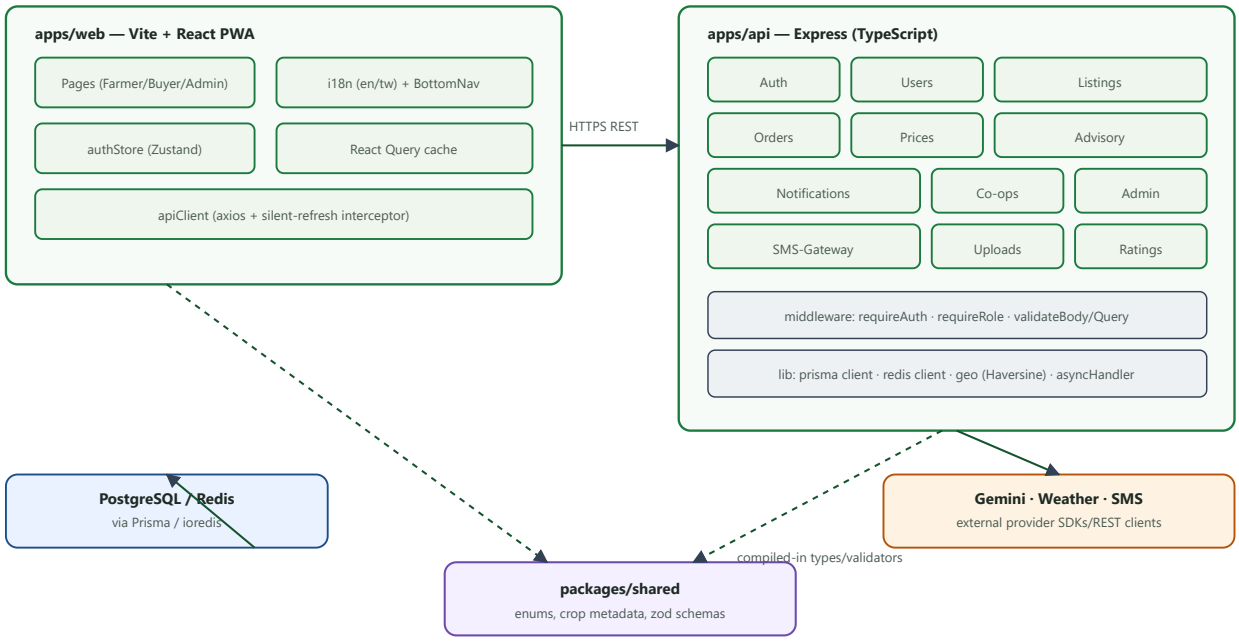

Figure 3.1 — Component diagram. Solid arrows are runtime calls; dashed arrows are compile-time dependencies on `packages/shared`, the single source of truth for enums (Role, OrderStatus, ListingStatus...) and zod validation schemas used by both apps.

4. Use-Case Diagram & Descriptions

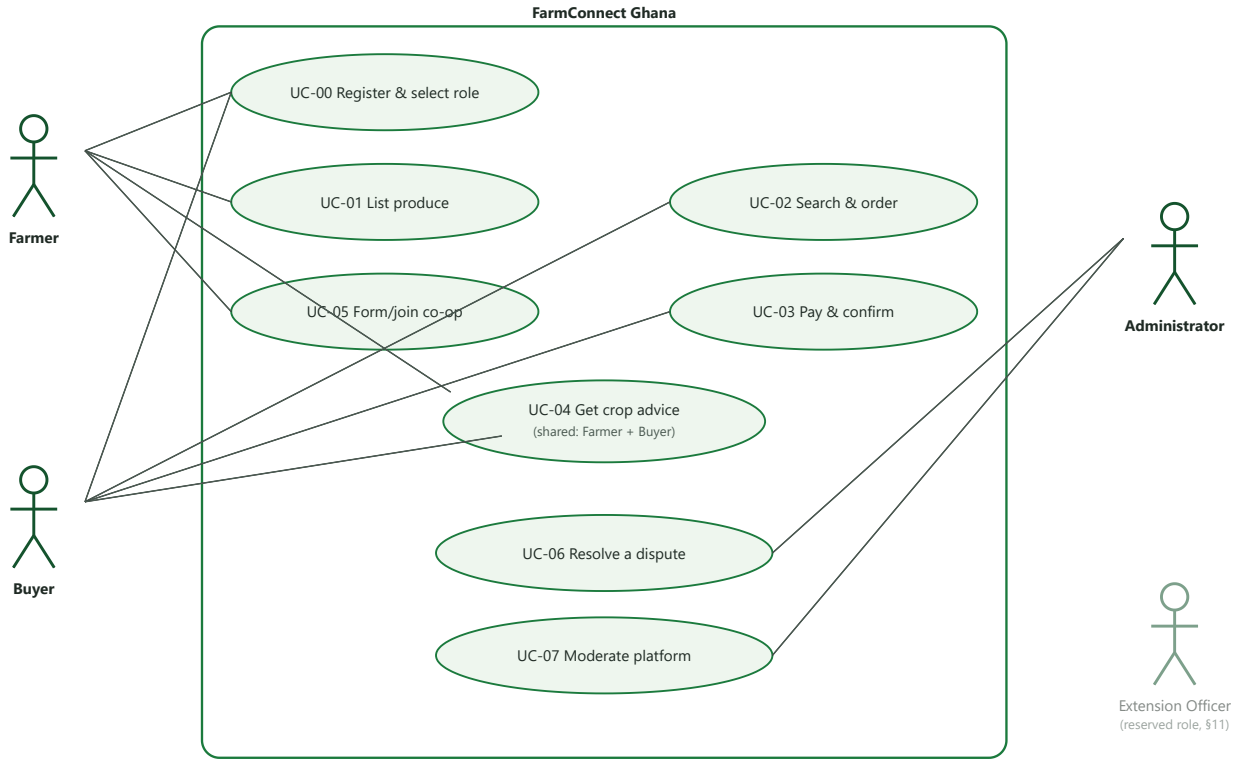

Figure 4.1 — Use-case diagram. UC-04 is shared by both Farmer and Buyer; UC-06 (dispute) involves Buyer, Farmer, and Admin but is drawn once and cross-referenced in the table below to avoid line clutter. The Extension Officer actor is shown at reduced opacity — reserved in the role model, not connected to any implemented use case (SRS §9.3).

Use case	Primary actor(s)	Preconditions → main flow → postcondition
UC-00	Farmer, Buyer	None → phone+OTP verified, role chosen (Farmer/Buyer only) → session issued (access+refresh JWT).
UC-01	Farmer	Mobile Money linked → crop/qty/price/photos/location submitted → listing `ACTIVE`, visible in marketplace search.
UC-02	Buyer	Listing `ACTIVE` → buyer searches/filters, opens detail, places order → order `pending`, listing `PENDING`.
UC-03	Buyer, Farmer, (Admin)	Order `pending` → buyer pays via own MoMo app, submits txn ID → farmer confirms/rejects → (optional) buyer disputes → admin resolves → buyer confirms delivery → order `delivered`, listing `SOLD`.
UC-04	Farmer, Buyer	User authenticated → sends text and/or photo → advisor (Gemini, weather-aware) replies → exchange persisted to chat history.

Use case	Primary actor(s)	Preconditions → main flow → postcondition
UC-05	Farmer	Not already in a co-op → create (become leader) or join via code (become member) → co-op membership active; listings may be attributed to it.
UC-06	Buyer, Farmer, Admin	Order `payment_rejected` → buyer raises dispute → order `disputed` → admin resolves (uphold payment/rejection) → order `paid` or `cancelled`, both notified.
UC-07	Admin	Admin authenticated → verify/suspend a user or force a listing's status → change applied immediately, affected user notified.

5. Class Diagram

A logical, object-oriented view of the core domain (attributes trimmed to the most salient fields — see §6 for the full field-level relational schema). Enumerations are shown as UML «enumeration» classes attached by dependency to the classes that use them.

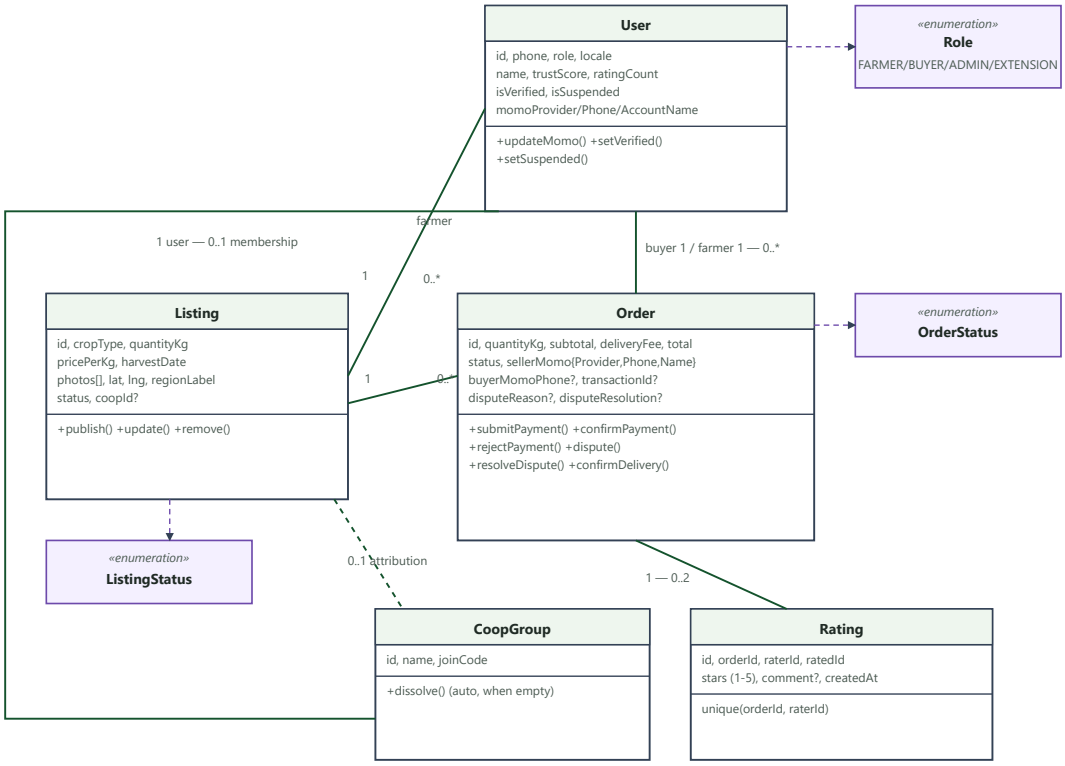

Figure 5.1 — Simplified class diagram. Full attribute lists, exact types, and every relation (including `Notification`, `ChatMessage`, `PriceSnapshot`, `SmsInboundLog`, and `CoopMember`) are given in the Entity-Relationship diagram and Database Design tables (§6), which is the authoritative field-level reference.

6. Entity-Relationship Diagram & Database Design

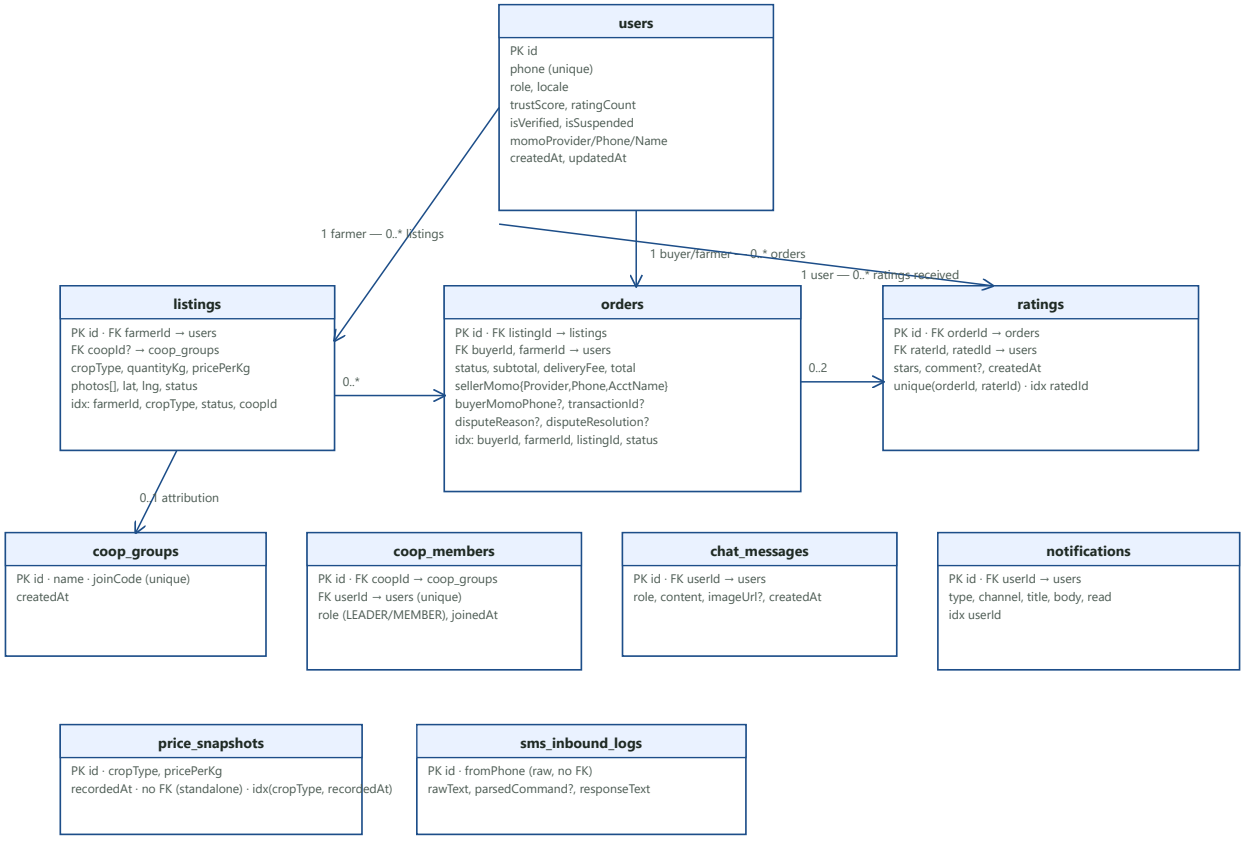

Figure 6.1 — Full entity-relationship diagram (all 10 tables). Only the most illustrative cardinalities are drawn as crow's-foot lines; every other foreign key (e.g. `coop_members.userId`, `chat_messages.userId`, `notifications.userId`) is stated as text inside its table above. `price_snapshots` and `sms_inbound_logs` are intentionally standalone time-series/audit tables with no foreign key at all.

Database Design Notes

- **Engine:** PostgreSQL 16, accessed exclusively through Prisma ORM (typed client, migration history in `apps/api/prisma/migrations/` — 7 migrations tracing the phase-by-phase schema growth from `init` to `add_phase6_ratings_coops_admin`).
- **Primary keys:** all tables use `cuid()` string IDs (collision-resistant, sortable-ish, no auto-increment leakage of row counts).
- **Money fields:** `Decimal(10,2)`, never floating point — `pricePerKg`, `subtotal`, `deliveryFee`, `total`.
- **Data integrity by constraint, not just application logic:** `users.phone` unique; `coop_members.userId` unique (enforces "at most one co-op per farmer" at the database level, not just in the service layer); `ratings` unique on `(orderId, raterId)` (enforces "one rating per rater per order").

- **Indexes:** added on every foreign key used in a hot lookup path (`farmerId` , `buyerId` , `status` , `cropType` , `userId`) — see per-table notes in Figure 6.1.
- **No PostGIS:** listings store plain `lat` / `lng` floats; radius search and sorting are computed in application code via a Haversine helper (`lib/geo.ts`) after a non-geo-filtered fetch — an explicit, documented simplification appropriate at capstone scale (SRS §9.8-adjacent).

7. Sequence Diagrams

7.1 Authentication — OTP login and token refresh

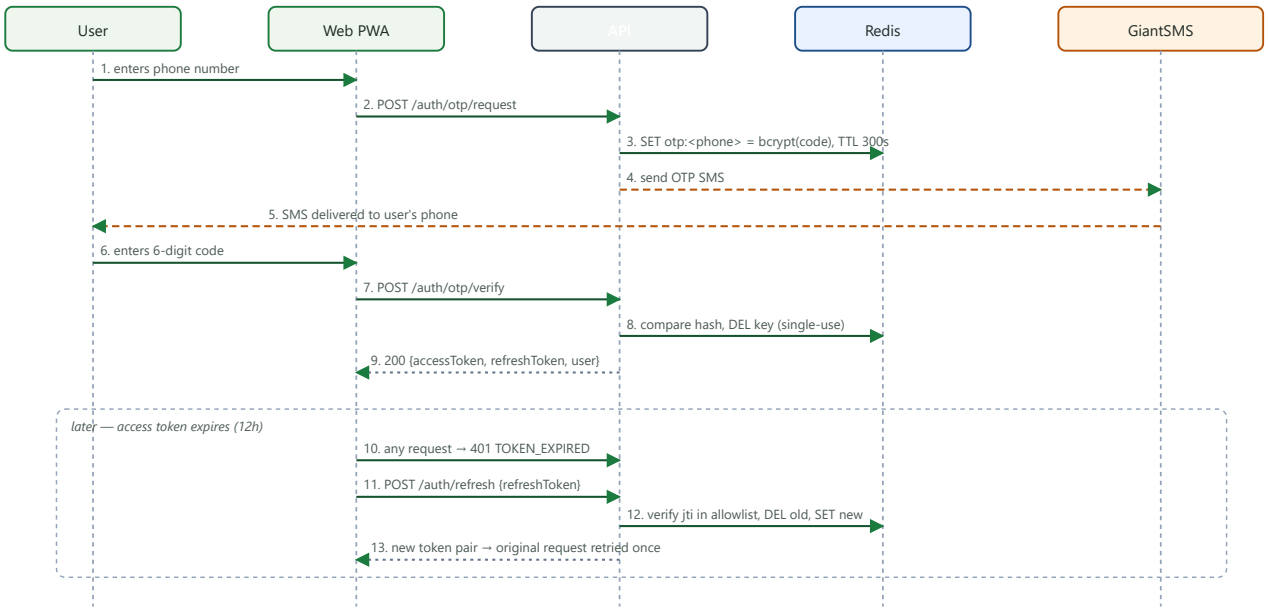

Figure 7.1 — OTP login followed by the single-use, rotating refresh-token flow. Non-production environments also return a plaintext `devCode` at step 3 to make demoing/testing possible without a live SMS credential.

7.2 Order lifecycle — manual Mobile Money reconciliation

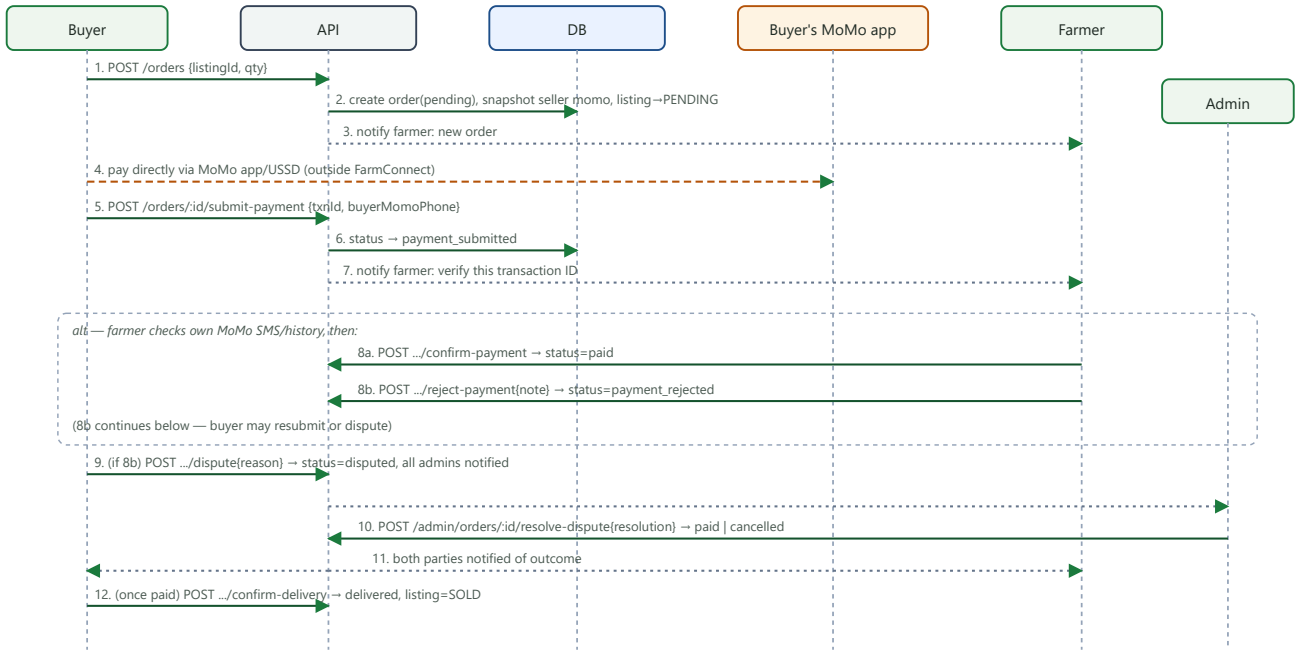

Figure 7.2 — End-to-end order + manual Mobile Money reconciliation, including the dispute branch (SRS FR-14). FarmConnect issues every notification but never touches the payment itself — step 4 happens entirely outside the system.

7.3 AI Advisory — weather-aware chat with photo pest identification

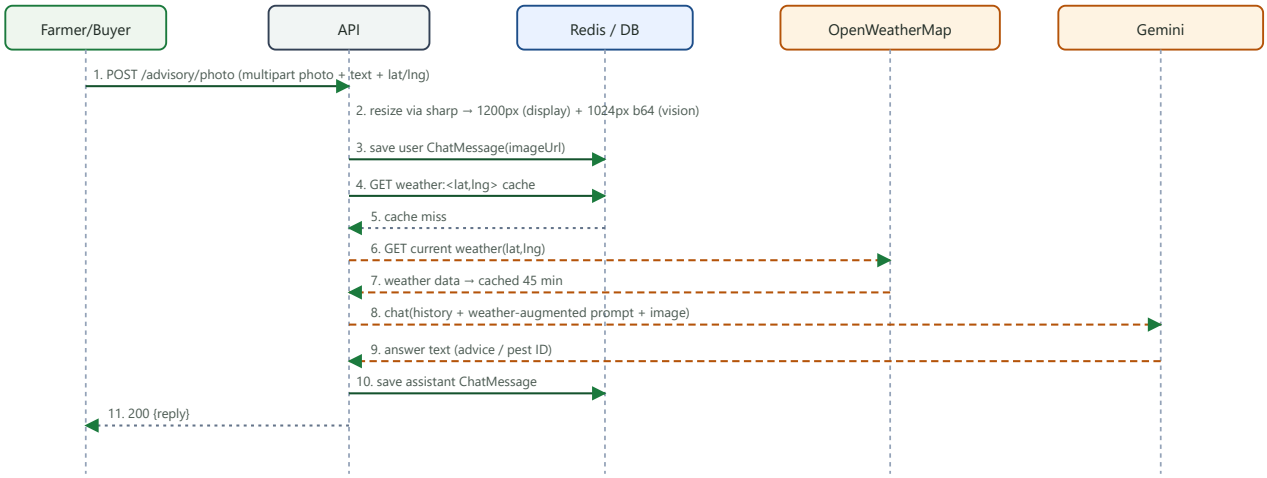

Figure 7.3 — Advisory chat with an attached photo. If `GEMINI_API_KEY` is unset the flow fails loudly at step 8 (503) rather than fabricating advice; if the weather step (4-7) fails or is unconfigured, the flow continues without weather grounding rather than blocking.

8. Order Lifecycle — State Diagram

The `Order.status` state machine, enforced by an exact-status check on every transition (an out-of-turn call returns `409 InvalidOrderState`). This is the process/activity view requested by the coursework brief, expressed as a state machine because that is the literal implementation.

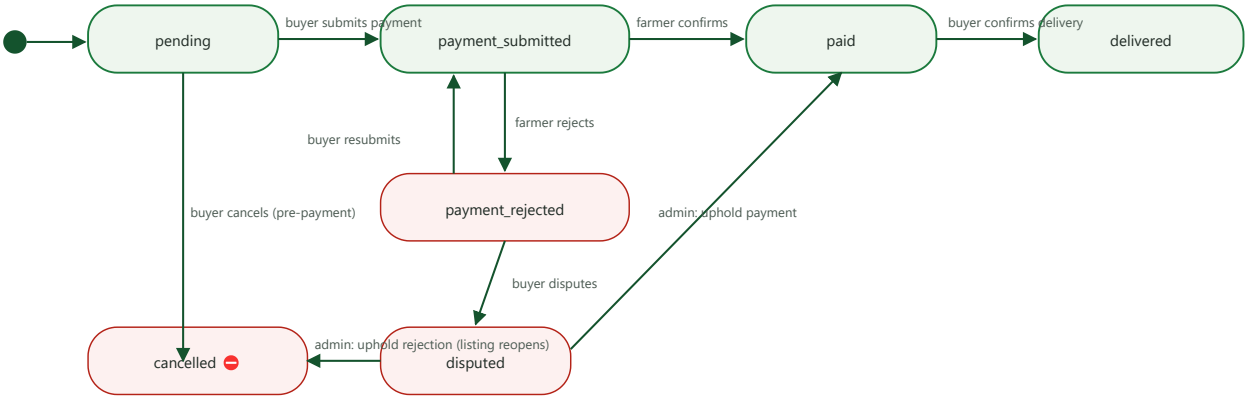

Figure 8.1 — Order state machine. Red rounded boxes are terminal/negative states (`cancelled`) or states requiring human intervention before they can resolve (`payment_rejected`, `disputed`). Reaching `delivered` also flips the parent `Listing.status` to `SOLD`; reaching `cancelled` reopens it to `ACTIVE`.

9. User Interface Wireframes

Low-fidelity wireframes of the actual delivered screens (not the early Figma-Make prototype, which was a copy/flow reference only and was never pixel-matched). Screen names and navigation structure are taken directly from `apps/web/src/routes.tsx` and `BottomNav.tsx`.

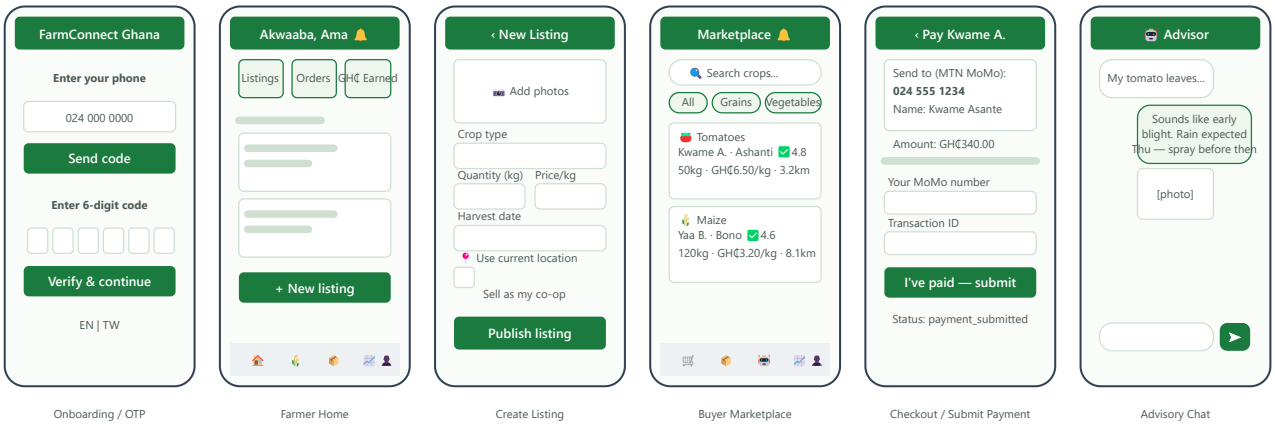

Figure 9.1 — Representative screens for both roles. The Admin Portal (`/admin/users`, `/admin/listings`, `/admin/disputes`) intentionally uses a different, desktop-oriented table layout with no bottom navigation — it is an internal staff tool, English-only (SRS §9.7), not the farmer/buyer-facing mobile experience shown here.

10. Technology Architecture

Layer	Choice	Why
Frontend	Vite + React 18 + TypeScript, Tailwind CSS, i18next, vite-plugin-pwa	Fast dev/build, installable offline-capable PWA with zero native-app-store friction, matches NFR-M2/U1.
Backend	Express 4 + TypeScript, Prisma ORM, ioredis	Simple, well-understood REST stack; Prisma gives type-safe migrations; module-per-domain layout satisfies NFR-M1.
Data	PostgreSQL 16, Redis 7 (Docker Compose locally)	Relational integrity for a transactional marketplace; Redis for anything ephemeral/rate-limited (OTP, refresh allowlist, weather cache).
Auth	Phone + OTP, JWT access (12h) + rotating refresh (30d) via Redis allowlist	No password to manage/leak; refresh rotation gives real revocation (logout) despite JWTs being otherwise stateless.
AI	Google Gemini (`gemini-flash-latest`)	Free developer tier with no per-end-user account needed — the only realistic choice for a chatbot shared by many unregistered-with-the-provider users.
Offline	Workbox runtime caching (NetworkFirst for API GETs, CacheFirst for photos)	Meets NFR-R3 — cached last-seen data on a dropped/poor connection, never masking staleness (OfflineBanner).
Monorepo	npm workspaces — `apps/web` , `apps/api` , `packages/shared`	One source of truth for enums/validation shared by both apps; independent build/test/deploy per workspace.

Full hosting/deployment topology, provider choices, and step-by-step deployment instructions are in the *Project Documentation* §Deployment and the standalone Deployment Guide.

11. Design Decisions Where the SRS Was Silent

FR-08 (ratings), FR-10 (co-ops), and FR-13 (admin) were each one sentence of intent in the v1.0 SRS, with no data model, workflow, or screen spec — the team had to design these from scratch during implementation. This section records the concrete decisions and the reasoning, for design-review traceability (cross-referenced from SRS §9.3/9.6).

11.1 Ratings & trust

Design choice: a rating is only possible once an order reaches `delivered` (you can't rate a transaction that didn't happen), exactly one rating per (order, rater) pair (enforced by a DB unique constraint, not just application logic), and the rated user's `trustScore` / `ratingCount` are recomputed inside the same database transaction as the insert — chosen specifically so the two numbers can never drift apart under concurrent writes.

11.2 Co-operative groups

Design choice: one co-op per farmer at a time (a simplifying constraint the SRS never ruled out), a short human-shareable join code instead of an approval workflow (lower friction for a rural, low-literacy user base), and listing-level attribution instead of building a separate bulk-order/negotiation subsystem the SRS never specified in enough detail to design safely. Leadership succession (earliest-joined member promoted, or dissolution if the leader was the sole member) was chosen as the simplest rule that avoids a leaderless or orphaned co-op.

11.3 Administration & moderation

Design choice: no self-service admin signup at all — the public role-selection endpoint only ever issues FARMER/BUYER, and an admin account is created by whoever controls the deployment via a one-line seed command, then logs in through the ordinary phone+OTP flow. This was chosen over a separate admin login screen/credential because it reuses a flow that is already tested and audited, and because the number of admins in a system like this is small and known in advance, not something that benefits from self-service.